When you first use VCR from your tests, it will record network requests to the real API together with their responses. Later test runs will use the recorded data instead of making real API calls.

In the next section, we'd like to show you what it's like to test with a high-fidelity fake—specifically, the StringIO class that ships with Ruby. With it, you can simulate I/O to the console, a disk file, the network, pipes, and more.

Faking I/O with StringIO

Long before Ruby on Rails came along, the first web applications were simple command-line scripts that wrote their content to the console. This Common Gateway Interface (CGI) architecture made it possible to build dynamic websites in nearly any language.[13] All you had to do was read your input from environment variables and write the resulting web page to stdout.

Here's a CGI script that functions as a simple little Ruby documentation server. If you were to hook this code up to a local web server and visit http://localhost/String/each, it would return a JSON array of all the String methods that begin with each: ["each_byte", "each_char", ...].

15-using-test-doubles-effectively/19/ruby_doc_server/lib/ruby_doc_server.rb

```
require 'json'

class RubyDocServer
  def initialize(output: $stdout)
    @output = output
  end

  def process_request(path)
    class_name, method_prefix = path.sub(%r{^/}, '').split('/')
    klass = Object.const_get(class_name)
    methods = klass.instance_methods.grep(/\A#{method_prefix}/).sort
    respond_with(methods)
  end

private

  def respond_with(data)
    @output.puts 'Content-Type: application/json'
    @output.puts
    @output.puts JSON.generate(data)
  end
end

if __FILE__.end_with?($PROGRAM_NAME)
  RubyDocServer.new.process_request(ENV['PATH_INFO'])
end
```

13. https://en.wikipedia.org/wiki/Common_Gateway_Interface